

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

«Программирование»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

Указатели и динамическая память.

Выполнил:

Студент группы N3150

Пахомов Владимир Юрьевич



Проверил:

Гирик А.В.

Санкт-Петербург

2024г.

Оглавление

Задание. Вариант 2-33.....	3
Make-файл.....	3
Примеры работы программы на различных исходных данных.....	4
Исходный текст программы с комментариями.....	5

Задание. Вариант 2-33

Способ представления матрицы: Одномерный массив, содержащий столбцы матрицы.

Задание: Назовём характеристикой столбца результат поразрядной операции «стрелка Пирса» между элементами столбца. Осуществить циклический сдвиг каждого столбца на количество единиц в старшем байте значения характеристики.

Тип элементов: uint32_t

Make-файл

```
.PHONY: all clean
```

```
APP=prg2vypN3150
```

```
CFLAGS=-Wall -Wextra -Werror -g
```

```
all: $(APP)
```

```
$(APP): $(APP).c
```

```
gcc -o $(APP) $(CFLAGS) $(APP).c
```

```
clean:
```

```
rm $(APP)
```

Примеры работы программы на различных исходных данных

При указании количества строк и столбцов матрицы программа заполняет матрицу случайными числами и производит операции над ней:

```
~/C_Workspace/prg2vypN3150 ➤ ./prg2vypN3150 3 3
Исходная матрица:
1870134121 493492401 681180891
565718660 82324014 1746488099
2049929734 723694568 478137384
Результат:
1870134121 493492401 681180891
1746488099 565718660 82324014
2049929734 723694568 478137384
```

При передаче опции -m программа просит ввести построчно все элементы матрицы и преобразование уже производится на матрице, которую ввёл пользователь:

```
~/C_Workspace/prg2vypN3150 ➤ ./prg2vypN3150 -m 2 2
Введите строки матрицы:
12 34
123 124
Исходная матрица:
12 34
123 124
Результат:
34 12
124 123
```

При передаче лишних аргументов, строк, не являющихся числами, при передаче отрицательных и вещественных чисел программа обрабатывает ошибку и выводит информацию о ней на экран:

```
~/C_Workspace/prg2vypN3150 ➤ ./prg2vypN3150 2
Usage: ./prg2vypN3150 [-m] число_строк число_столбцов
~/C_Workspace/prg2vypN3150 ➤ ./prg2vypN3150 -m 3
Ошибка: '-m' не является целым числом.
~/C_Workspace/prg2vypN3150 ➤ ./prg2vypN3150 -g 2 5
Ошибка: опция '-g' не поддерживается.
~/C_Workspace/prg2vypN3150 ➤ ./prg2vypN3150 2 -3
Ошибка: аргументы должны быть положительными числами.
~/C_Workspace/prg2vypN3150 ➤ ./prg2vypN3150 prg 3
Ошибка: 'prg' не является целым числом.
~/C_Workspace/prg2vypN3150 ➤ ./prg2vypN3150 2 21.1
Ошибка: '21.1' не является целым числом.
~/C_Workspace/prg2vypN3150 ➤
```

Исходный текст программы с комментариями

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

#include <stdint.h>
#include <ctype.h>
#include <time.h>
#include <unistd.h>

// Функция проверки, является ли строка числом
int check_number(const char *str) {
    char *endptr;
    strtol(str, &endptr, 10);
    if (*endptr != '\0') return 0;
    else return 1;
}

// Вывод матрицы
void print_matrix(uint32_t *matrix, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrix[i * cols + j]);
        }
        printf("\n");
    }
}

// Характеристика столбца
uint32_t column_characteristic(uint32_t *matrix, int rows, int cols, int col) {
    uint32_t result = matrix[col];
    for (int i = 1; i < rows; i++) {
        result = ~(result | matrix[i * cols + col]); // Стрелка Пирса для всех элементов столбца
    }
    return result;
}

// Циклический сдвиг столбца
void cyclic_shift_column(uint32_t *matrix, int rows, int cols, int col, int shift) {
    shift = shift % rows;
    uint32_t temp[rows];
    for (int i = 0; i < rows; i++) {
        temp[i] = matrix[i * cols + col];
    }
    for (int i = 0; i < rows; i++) {
        matrix[i * cols + col] = temp[(i + shift) % rows];
    }
}

int main(int argc, char *argv[]) {
```

```

// Проверка запуска с переменной среды, включающей отладочный вывод.
// Пример запуска с установкой переменной LAB2DEBUG в 1:
// $ LAB2DEBUG=1 ./prg2abcNXXXXX 5 5
char *DEBUG = getenv("LAB2DEBUG");
if (DEBUG) {
    fprintf(stderr, "Включен вывод отладочных сообщений\n");
}

int manual_input = 0;
int arg_index = 1;
char *endptr;

if ((argc < 3) || (argc > 4)) {
    fprintf(stderr, "Usage: %s [-m] число_строк число_столбцов\n", argv[0]);
    return EXIT_FAILURE;
}
if (argc == 4) {
    if (strcmp(argv[1], "-m") == 0) {
        manual_input = 1;
        arg_index++;
    } else {
        fprintf(stderr, "Ошибка: опция '%s' не поддерживается.\n", argv[1]);
        return EXIT_FAILURE;
    }
}
if (argc - arg_index < 2) {
    fprintf(stderr, "Usage: %s [-m] число_строк число_столбцов\n", argv[0]);
    return EXIT_FAILURE;
}
if (!check_number(argv[arg_index])) {
    fprintf(stderr, "Ошибка: '%s' не является целым числом.\n", argv[arg_index]);
    return EXIT_FAILURE;
}
if (!check_number(argv[arg_index + 1])) {
    fprintf(stderr, "Ошибка: '%s' не является целым числом.\n", argv[arg_index + 1]);
    return EXIT_FAILURE;
}

int rows = strtol(argv[arg_index], &endptr, 10);
int cols = strtol(argv[arg_index + 1], &endptr, 10);

if (rows <= 0 || cols <= 0) {
    fprintf(stderr, "Ошибка: аргументы должны быть положительными числами.\n");
    return EXIT_FAILURE;
}

uint32_t matrix[rows * cols];
srand(time(NULL) + getpid());

if (manual_input) { // Если -m передан, то матрицу вводим вручную

```

```

    printf("Введите строки матрицы:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (!scanf("%u", &matrix[i * cols + j])) {
                fprintf(stderr, "Ошибка: введенное значение не является целым положительным
числом.\n");
                return EXIT_FAILURE;
            }
        }
    }
} else { // Если -m не передан, то генерируем случайные числа
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i * cols + j] = rand() % UINT32_MAX; // Заполнение случайными числами от 0
до
        }
    }
}

printf("Исходная матрица:\n");
print_matrix(matrix, rows, cols);
// Транспонируем матрицу, чтобы массив состоял из столбцов
uint32_t trans_matrix[rows * cols];
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        trans_matrix[j * rows + i] = matrix[i * cols + j];
    }
}
// Считаем характеристику для каждого столбца и сдвигаем столбцы
for (int j = 0; j < cols; j++) {
    uint32_t characteristic = column_characteristic(trans_matrix, rows, cols, j);
    int shift = (characteristic >> (sizeof(uint32_t) - 1)*8) & 0xFF; // Считаем единицы в
старшем байте
    cyclic_shift_column(trans_matrix, rows, cols, j, shift);
}

// Полученную матрицу опять транспонируем для вывода ответа
uint32_t res_matrix[rows * cols];
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        res_matrix[j * rows + i] = trans_matrix[i * cols + j];
    }
}
printf("Результат:\n");
print_matrix(res_matrix, cols, rows);

return EXIT_SUCCESS;
}

```